Case 2 Lösungsdokumentation

Automatisierung der Versandentscheidung

MODERATOR
Roberto Panizza

AUTOREN
Roberto Panizza, Loris Trifoglio

BASIS
Case 1 – Versandprozess

Inhaltsverzeichnis

1 Abgabe

2 Ausgangslage und Randbedingungen

3 Lernfragen und Entscheidungen

3.1 Platzierung der Fachentscheidung

3.2 Drools vs. BPMN-Gateways

3.3 Protokollierung

3.4 Monitoring

4 Zielarchitektur

4.1 High-Level-Übersicht

4.2 Verantwortung pro Komponente

5 Fachlicher Entscheidungsfluss

6 Regelstrategie

6.1 Bekannte Regeln

6.2 Operative Übersetzung

7 Rückgabevertrag des Decision Workers

8 BPMN-Anpassungen gegenüber Case 1

9 Implementierung

9.1 Modulstruktur

9.2 Kernklassen

10 Datenmodell MySQL 1

11 Sicherheit: SQL-Injection-Schutz

12 Monitoring-Konzept

13 Umsetzungsplan

14 Geplante Artefakte

15 Fazit

1 Abgabe

Unsere geplante Abgabe für Case 2 besteht aus:

- angepasstem BPMN-Prozess auf Basis von `case1/case1.bpmn`
- Spring-Boot-App mit zwei External-Task-Workern für Versandentscheidung und Speditionsaufruf
- Drools-Decision-Table für die bekannten Versandregeln
- Protokollierung der Entscheidungen in `MySQL 1`
- kurze Monitoring-Konzeptbeschreibung für die spätere Erweiterung
- Dokumentation der Architektur und des Umsetzungsplans

GitHub Repository: https://github.com/lto-oro/it-architecture

2 Ausgangslage und Randbedingungen

Case 2 baut direkt auf dem teilautomatisierten Versandprozess aus Case 1 auf. In Case 1 wird der Versandauftrag in Camunda orchestriert und der bestehende Shipping Service über einen External Task angebunden. Der in Case 2 adressierte manuelle Teil ist die fachliche Versandentscheidung, die laut Ausgangslage derzeit als undokumentierte Ergänzung von Schritt 2 erfolgt.

Die Problemstellung aus *Case 2 Ausgangslage.pdf* verlangt:

- Automatisierung der einfachen, bekannten Versandregeln
- klare Trennung der Verantwortlichkeiten für spätere Erweiterbarkeit
- Vorbereitung auf eine spätere AI-gestützte Entscheidungslogik
- Protokollierung der Entscheidungen
- Berücksichtigung eines späteren Monitorings

Für Case 2 stehen im vorhandenen Setup konkret die folgenden Systeme zur Verfügung:

- **Camunda**
- **Shipping Service**

- **MySQL 1**

Nicht als verfügbare Infrastruktur für Case 2 sichtbar sind: dedizierte Business-Rule-Infrastruktur, Prometheus/Grafana sowie weitere Versandprovider-Services.

Daraus folgt für die Zielarchitektur:

- die Rule Engine muss als eigene fachliche Komponente innerhalb der Case-2-Lösung bereitgestellt werden
- die Entscheidungsprotokollierung erfolgt auf `MySQL 1`
- Monitoring wird nur vorbereitet, aber nicht als eigene Plattform umgesetzt
- die bestehende Shipping-API bleibt bestehen und wird durch einen in `case2` integrierten Worker auf Topic `group4_rest` genutzt

3 Lernfragen und Entscheidungen

3.1 Wo wird die Fachentscheidung platziert?

Nicht in Camunda selbst. Camunda bleibt für Orchestrierung und Prozesszustand zuständig. Die Versandentscheidung wird in eine eigene Spring-Boot-Komponente ausgelagert. Dadurch bleibt die Entscheidungslogik austauschbar und später durch AI ersetzbar oder ergänzbar.

3.2 Warum Drools und nicht reine BPMN-Gateways?

Die Regeln sind fachlich volatil und sollen später wachsen. Eine tabellenbasierte Rule Engine ist für diesen Anwendungsfall besser wartbar als eine stetig wachsende Menge an BPMN-Ausdrücken. BPMN soll nur das Prozessrouting abbilden, nicht die komplette Fachlogik.

3.3 Wo wird protokolliert?

In `MySQL 1`, weil dieses System im Case-1/2-Bereich laut System Overview bereits vorhanden ist. Damit gibt es eine persistente Audit-Spur für spätere Analyse, Nachvollziehbarkeit und AI-Training.

3.4 Wie wird Monitoring berücksichtigt?

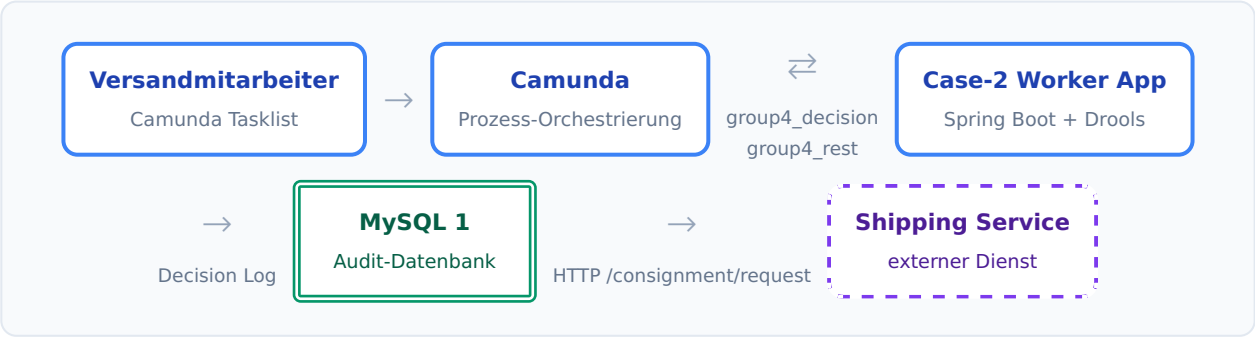
Da keine Monitoring-Plattform im Bild für Case 2 vorhanden ist, wird nur die technische Anschlussfähigkeit vorbereitet:

- Spring Boot Actuator
- strukturierte Logs mit Decision ID und Order ID
- persistente Entscheidungsprotokolle in MySQL

Später kann darauf Prometheus/Grafana aufsetzen, ohne die Fachlogik neu schneiden zu müssen.

4 Zielarchitektur

4.1 High-Level-Übersicht



4.2 Verantwortung pro Komponente

Camunda

- Orchestrierung des Gesamtprozesses
- Benutzerinteraktion über Forms
- Routing zwischen automatisiertem und manuellem Pfad

Case-2 Worker App

- Auswertung der Versandregeln
- Kapselung der Rule Engine
- Abholung und Bearbeitung beider Camunda External Tasks (`group4_decision` und `group4_rest`)
- Persistente Protokollierung der Entscheidung

MySQL 1

- Audit-Log aller Entscheidungen
- Grundlage für spätere Auswertung und AI-Training

Shipping Service

- bleibt unverändert aus Case 1 bestehen
- wird nur für automatisch freigegebene Standard-Speditionsfälle genutzt

5 Fachlicher Entscheidungsfluss

Der Prozess für Case 2 wird wie folgt erweitert:

1. Benutzer erfasst bzw. ergänzt die Versanddaten in Camunda.
2. Camunda erzeugt einen External Task *Versandentscheidung ermitteln*.
3. Der **Decision Worker** holt den Task ab und wertet die Regeln mit Drools aus.
4. Die Entscheidung wird inklusive Eingabedaten, Regel-ID und Begründung in `MySQL 1` protokolliert.
5. Camunda verzweigt anhand des Rückgabewerts:
   - `STANDARD_SPEDITION_AUTOMATED` → Weiter mit *Spedition beauftragen* (Topic `group4_rest`)
   - `MANUAL_REVIEW_REQUIRED` → manuelle Bearbeitung mit Entscheidungshinweis

> **Hinweis:** Da im System Overview für Case 2 nur der bestehende Shipping Service sichtbar ist, wird ausschließlich der Standard-Speditionspfad vollautomatisch weitergeführt. Andere erkannte Fälle werden fachlich korrekt klassifiziert, aber in einen manuellen Bearbeitungsschritt übergeben.

6 Regelstrategie

6.1 Bekannte Regeln aus der Aufgabenstellung

- **Argentinien, 60–500 kg:** spezielle Spedition
- **Argentinien, < 60 kg:** normale Post
- **Argentinien, > 500 kg:** manuelle Behandlung
- **Japan, bis 200 kg:** Luftfracht
- **Russische Föderation:** manuelle Prüfung wegen Sanktionen
- **Schweiz und Deutschland:** normale Spedition

6.2 Operative Übersetzung

Land / Bedingung	Entscheidungsresultat	Auto?	Begründung
Schweiz oder Deutschland	`STANDARD_SPEDITION_AUTOMATED`	✓ Ja	Shipping Service verfügbar
Russische Föderation	`MANUAL_REVIEW_REQUIRED`	✗ Nein	Sanktionsprüfung notwendig
Argentinien < 60 kg	`MANUAL_REVIEW_REQUIRED`	✗ Nein	Post-Schnittstelle nicht vorhanden
Argentinien 60–500 kg	`MANUAL_REVIEW_REQUIRED`	✗ Nein	Spezialspedition nicht vorhanden
Argentinien > 500 kg	`MANUAL_REVIEW_REQUIRED`	✗ Nein	explizit fallspezifisch
Japan ≤ 200 kg	`MANUAL_REVIEW_REQUIRED`	✗ Nein	Luftfracht-Schnittstelle fehlt
Sonst	`MANUAL_REVIEW_REQUIRED`	✗ Nein	keine freigegebene Regel vorhanden

> Diese Übersetzung ist bewusst konservativ: Es werden nur Fälle automatisiert, für die ein echter Ausführungspfad vorhanden ist.

7 Rückgabevertrag des Decision Workers

Eingabe (Camunda-Prozessvariablen)

```
{
  "order_nr":          "4711",
  "client_nr":         "100023",
  "delivery_country":  "CH",
  "delivery_address":  "Musterstrasse 1, 4051 Basel",
  "weight":            120,
  "phone":             "+41611234567",
  "mail":              "kunde@example.org"
}
```

Ausgabe (zurückgeschriebene Variablen)

```
{
  "decision_id":                 "0c5d1d4e-8f5b-4c4b-b7c0-5d73e98b8d29",
  "decision_outcome":            "STANDARD_SPEDITION_AUTOMATED",
  "decision_auto_continue":      true,
  "decision_recommended_channel":"STANDARD_SPEDITION",
  "decision_rule_id":            "CH_DE_STANDARD_SPEDITION",
  "decision_rule_version":       "v1",
  "decision_reason":             "Lieferungen in die Schweiz und nach Deutschland
                                  werden über die normale Spedition abgewickelt."
}
```

8 BPMN-Anpassungen gegenüber Case 1

Das bestehende Modell aus Case 1 wird nicht ersetzt, sondern gezielt erweitert.

Neue oder angepasste Prozessdaten

- `delivery_country` (neu – explizites Feld erforderlich)
- alle `decision_*`-Variablen gemäss Rückgabevertrag

> **Wichtig:** `delivery_country` existiert in `case1.bpmn` noch nicht. Im zweiten Formular werden nur `delivery_address`, `phone` und `mail` erfasst. Für verlässliche Regelverarbeitung ist ein explizites Feld vorzuziehen.

Neue Prozessschritte

1. Nach *Formular A38 mit weiteren Informationen ergänzen* wird ein neuer Service Task eingefügt: **Versandentscheidung ermitteln**
2. Danach folgt ein Gateway: **Automatische Weiterverarbeitung möglich?**
3. Bei `decision_auto_continue == true`: weiter zu *Spedition beauftragen*
4. Andernfalls: weiter zu *Manuelle Versandentscheidung bearbeiten*

Erwartete BPMN-Struktur

```
Bearbeitung Kundenauftrag
  → Formular A38 ergänzen
  → Versandentscheidung ermitteln
  → Gateway: decision_auto_continue?
     → Ja:   Spedition beauftragen → Email → Ablagefach P
     → Nein: Manuelle Versandentscheidung
               → ggf. Hotline / externe Bearbeitung
               → Email → Ablagefach P
```

9 Implementierung

9.1 Modulstruktur

```
case2/
  src/main/java/.../api/
  src/main/java/.../service/
  src/main/java/.../rules/
  src/main/java/.../persistence/
  src/main/resources/rules/shipping-rules.drl
  src/main/resources/rules/shipping-rules.csv
```

9.2 Kernklassen

Klasse	Aufgabe
`ShippingDecisionService`	Fachliche Orchestrierung der Entscheidung
`DroolsRuleEvaluator`	Ausführung der Decision Table
`DecisionExternalTaskWorker`	Camunda-Integration für Topic `group4_decision`
`SpeditionExternalTaskWorker`	Camunda-Integration für Topic `group4_rest` und HTTP-Aufruf des Shipping Service
`DecisionLogRepository`	Persistenz in MySQL 1 (Tabelle `shipping_decision_log`)

10 Datenmodell MySQL 1

Vorgeschlagene Tabelle `shipping_decision_log`:

Feld	Typ	Zweck
`id`	VARCHAR(36)	Technische Decision ID (UUID)
`created_at`	TIMESTAMP	Zeitpunkt der Entscheidung
`order_nr`	VARCHAR	Fachlicher Bezug
`client_nr`	VARCHAR	Fachlicher Bezug
`delivery_country`	VARCHAR(2)	Regelinput
`delivery_address`	TEXT	Regelinput / Nachvollziehbarkeit
`weight_kg`	INT	Regelinput
`outcome`	VARCHAR	Fachliches Resultat
`auto_continue`	BOOLEAN	Routing-Hinweis für BPMN
`recommended_channel`	VARCHAR	z.B. `STANDARD_SPEDITION`
`rule_id`	VARCHAR	Getroffene Regel
`rule_version`	VARCHAR	Version der Tabelle
`reason`	TEXT	Fachliche Begründung
`payload_json`	JSON / TEXT	Kompletter Input für spätere Analyse

11 Sicherheit: SQL-Injection-Schutz

Die Persistenz wird ausschließlich über parametrisierte SQL-Statements realisiert:

- `DecisionLogRepository` verwendet ein `PreparedStatement` mit Platzhaltern (`?`) und typisierter Parameterbindung.
- Es wird keine SQL-String-Konkatenation mit Benutzereingaben durchgeführt.
- `payload_json` wird strukturiert per Jackson serialisiert und als Parameter gebunden.

12 Monitoring-Konzept

Da im vorhandenen Setup keine dedizierte Monitoring-Infrastruktur sichtbar ist, wird für diesen Case nur die Anschlussfähigkeit geschaffen:

- Spring Boot Actuator aktivieren
- Health-Endpoint für Service und Datenbank
- Strukturierte Logs mit `decisionId`, `orderNr`, `outcome`

Geplante Metriken:

- Anzahl Entscheidungen gesamt
- Anteil automatische Weiterverarbeitung
- Anteil manueller Fälle
- Fehler bei Rule-Auswertung
- Fehler bei Datenbankpersistenz

> Später können diese Daten durch Prometheus/Grafana konsumiert werden, ohne dass sich die Kernarchitektur ändert.

13 Umsetzungsplan

Arbeitspaket 1 – Case-2-Projekt aufsetzen

- Neues Spring-Boot-Projekt in `case2` anlegen
- Abhängigkeiten: Spring Web, Spring JDBC, MySQL Driver, Drools, Actuator
- Basis-Config für `MySQL 1` und Logging

Arbeitspaket 2 – Decision API definieren

- Eingabe- und Ergebnisobjekte für die Regelentscheidung festlegen
- External-Task-Worker für Camunda implementieren
- Validierung für `delivery_country`, `weight` und Pflichtfelder

Arbeitspaket 3 – Drools-Regeln umsetzen

- Ausführbare Drools-Regeln in `shipping-rules.drl` anlegen
- Tabellarische Regelspezifikation in `shipping-rules.csv` pflegen
- Regeln mit allen bekannten Fällen modellieren
- Versionierung der Regeln vorsehen
- Unit-Tests für jede Regelkombination schreiben

Arbeitspaket 4 – Entscheidungsprotokollierung implementieren

- JDBC-Repository (`DecisionLogRepository`) anlegen
- Migration oder initiales SQL für Tabelle `shipping_decision_log`
- Persistenz nach jeder erfolgreichen Regelentscheidung
- SQL-Injection-Schutz durch Prepared Statements und Parameterbindung

Arbeitspaket 5 – BPMN aus Case 1 erweitern

- Neues BPMN für Case 2 ableiten
- External Service Task *Versandentscheidung ermitteln* einfügen
- Gateway für `decision_auto_continue`
- Formular A38 um `delivery_country` erweitern
- Manuellen Task für nicht automatisch ausführbare Entscheide modellieren

Arbeitspaket 6 – Integration mit bestehendem Versandpfad

- Nur für `STANDARD_SPEDITION_AUTOMATED` in den bestehenden Aufruf *Spedition beauftragen* verzweigen
- Für alle anderen Fälle die empfohlene Versandart im manuellen Task anzeigen

Arbeitspaket 7 – Observability und Rest-Risiken

- Actuator-Endpunkte prüfen
- Strukturierte Logs verifizieren
- Dokumentieren, welche Fälle noch nicht vollautomatisiert sind, weil passende Provider-Schnittstellen fehlen

14 Geplante Artefakte

Artefakt	Beschreibung
`case2` Spring-Boot-Service	Vollständige Worker-Applikation inkl. Drools-Integration
`case2` BPMN-Datei	Erweitertes Prozessmodell auf Basis von case1.bpmn
Drools-Decision-Table	`shipping-rules.drl` und `shipping-rules.csv`
SQL / JDBC	Schema und Repository für `shipping_decision_log`
Lösungsdokumentation	Dieses Dokument

15 Fazit

Die konkrete Zielarchitektur für Case 2 ist:

- **Camunda** für Orchestrierung
- **Case-2 Worker-App** für Regelentscheidung, Speditionsaufruf und Protokollierung
- **MySQL 1** für Audit und spätere AI-Grundlage
- **Shipping Service** nur für Standard-Speditionsfälle

Damit wird die Fachentscheidung sauber vom Prozess getrennt, auf die vorhandene Umgebung Rücksicht genommen und ein belastbarer technischer Pfad für spätere Erweiterungen geschaffen.